

A Generalisable Pipeline for Backporting ROCm to Obsolete AMD Hardware

Executive summary for technical mentorship

Core thesis

Build a reusable, benchmarked recovery pipeline for dropped AMD GPU support in ROCm, so older hardware can be revived without rediscovering the same failure modes from scratch.

Source-constrained summary based on the provided project draft; no funding language included.

Why this matters

Problem in one sentence

ROCm is the only credible open-source alternative to CUDA for GPU compute, but AMD periodically drops support for older architectures.

Capable hardware becomes software-stranded rather than physically obsolete.

This is especially acute for cost-constrained users relying on Ryzen APUs and Vega-class GPUs.

The gap is not a hardware failure; it is a missing migration path.

What the project targets

dropped GPU support
fragmented community fixes
missing reproduction steps
unclear performance cost

The gap: support exists in fragments, not as a workflow

Current reality

- scattered forum posts and issue threads
- ad hoc workarounds
- no single end-to-end reference
- little evidence about trade-offs

What is missing

- reproducible steps
- benchmarked comparisons
- clear failure boundaries
- a path others can reuse

Research gap

Existing recovery attempts are not systematised, not benchmarked, and not reproducible.

Proposed approach: a decision tree for recovery



Key idea

The project is a systematic pipeline, not a one-off workaround: each strategy isolates a different failure layer and each successful configuration is measured.

Recovery strategies under test

Community baseline

Environment-level spoofing and existing workarounds. Measures how far unsupported hardware runs before any rebuild.

Mixed-version stacks

ROCm userspace and AMDGPU kernel version decoupling. Shows whether failures come from driver ABI, userspace, or support boundaries.

Containerised vendor stacks

AMD Docker images across ROCm versions. Tests whether containers preserve the last usable stack or recover lost support.

Source builds

Rebuild PyTorch and ROCm components with explicit targets. Checks whether compilation alone restores missing functionality.

Design principle

Start with the least invasive path, then move toward costlier interventions only when they add diagnostic value.

Scope: two representative targets, one reusable method

Initial hardware targets

gfx90c — Ryzen 5 5650U / Cezanne
APU / Vega 7

gfx902 — Ryzen 5 2200G / Raven Ridge
/ Vega 8

Reference environments from the draft

Fedora 43 on the 5650U system

RHEL 9.5 on the 2200G desktop

system ROCm 6.4.4 on the current host
setup

Why these two

laptop and desktop form factors
same architectural family, different sys-
tems
useful cross-check for generalisation

Generalisation goal

The first traversal is about these two
targets; the methodology is intended to
transfer to other dropped AMD
architectures later.

How success will be measured

Evaluation at every stage

- direct HIP execution
- PyTorch runtime behavior
- performance relative to a working baseline
- reproducibility across systems and versions

Initial observation from the draft

The current gfx900 spoof path for gfx90c reportedly reaches about 50% of AMD's published figures for the Vega 7.

Comparison anchors in the draft

- latest stable ROCm for the target
- AMD's published theoretical figures
- observed results from the current workaround path

Retained because it appears explicitly in the draft and motivates the benchmarking plan.

What has already been established

Post 1: context and motivation

Frames the ROCm ecosystem, the hardware target, and the case for running modern ML tooling on dropped GCN5 hardware.

Post 2: installation and problem identification

Shows native ROCm 6.4.4 on Fedora, successful HIP execution, and the central blocker: missing gfx90c kernels in bundled rocBLAS for PyTorch.

Status signal

The hardware compute stack is functional; the unresolved issue is support continuity in ROCm userspace and packaged kernels.

Execution path: from diagnosis to validation

1. Baseline

Test existing workarounds and identify the first failure layer.

2. Decouple

Mix ROCm userspace and AMDGPU versions to isolate compatibility boundaries.

3. Containerise

Compare AMD Docker images across ROCm versions on the same host stack.

4. Rebuild

Compile PyTorch and, where needed, ROCm components with explicit architecture targeting.

5. Validate

Re-run successful configurations on gfx902 to test generalisation.

6. Document

Publish the reproduction steps, benchmarks, and failure modes openly.

Why mentorship would matter

What I am trying to validate

Whether dropped AMD GPU support can be recovered through spoofing, stack separation, containerisation, and selective source builds — with benchmarks at every stage.

tighten the technical methodology
pressure-test the feasibility boundaries
avoid dead ends that look promising but do not scale
turn one hardware case into a reusable playbook

Mentorship ask

Review the technical approach, benchmark discipline, and recovery strategy ordering so the work stays rigorous and generalisable.

Closing line

The objective is not just to make one GPU work; it is to build a method that others can apply when ROCm drops the next architecture.